
Clinical Graph-JEPA: Predictive Patient-State Knowledge Graphs for Cognitive Decision Support

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Clinical text contains rich evidence about patient state, but converting it into reliable,
2 structured knowledge graphs remains difficult because extraction errors, ontology
3 mismatch, missing relations, and temporal ambiguity propagate into downstream
4 decision support systems. We propose a framework for clinical knowledge graph
5 construction that combines multi-agent extraction, ontology-aware normalization,
6 critic-based verification, and Graph-JEPA-based latent refinement. The central idea
7 is to treat the clinical knowledge graph as a predictive patient-state representation
8 rather than a static extraction artifact. Given clinical notes or transcripts, the system
9 constructs an initial graph, verifies extracted entities and relations against clinical
10 constraints, and learns to predict masked or missing graph regions from observed
11 context. This enables the model to capture latent dependencies among symptoms,
12 medications, behaviors, labs, and diagnoses. We evaluate the framework on clinical
13 text datasets using extraction quality, graph completion, downstream prediction,
14 and retrieval-augmented reasoning metrics.

15 1 Introduction

16 Structured clinical data (diagnosis, medication, and procedure tables) records *what* happened to a
17 patient but not the clinician’s *reasoning*: that a β -blocker was started *for* the patient’s atrial fibrillation,
18 that a presenting chest pain *indicated* the acute coronary syndrome that was ultimately worked up.
19 That reasoning lives in the free-text note. A large language model (LLM) can extract it as graph
20 edges, but those inferred edges are exactly the ones with no table-level provenance, so a drafted
21 clinical KG is a mixture of certain structural facts and uncertain inferred relations of widely varying
22 quality. We improved missing critical relationships (INDICATES, CONFIRMS) recovery by 34-60

23 We take the position that the right object to learn is not a one-shot extractor but a *refiner*: a model that,
24 given an already-drafted, evidence-scored graph, predicts which edges belong. We instantiate this as
25 a **Graph-JEPA world model** — a self-supervised graph encoder trained by masked-latent prediction,
26 with a lightweight readout that recovers held-out edges. Two findings drive the design. First, notes
27 are *partial*: in a longitudinal MIMIC-IV cohort only 64.4% of admissions carry a discharge note, so
28 a deployable system cannot assume one is present. We therefore frame the refiner as two explicit
29 options — a *note-free* model (Option A) that uses no notes and a *note-augmented* model (Option B)
30 that uses the notes that are available — and evaluate both. Second, *how* the note is injected matters
31 more than *whether* it is: a diffuse, admission-global note vector slightly hurts, whereas localising the
32 same vector onto the entities the note grounds nearly doubles inferred-edge recovery.

33 **Contributions.** (1) A consolidated, evidence-scored clinical-graph construction that fuses a de-
34 terministic MIMIC-IV backbone with LLM-inferred relations and an 18-signal per-edge score. (2)
35 A Graph-JEPA world model that refines these drafts, evaluated by a leakage-free leave-one-out
36 edge-recovery protocol. (3) Two deployment options for partial notes — **Option A** (note-free) and

37 **Option B** (note-augmented) — reported side by side. (4) The empirical result that note content must
38 be injected *locally* (entity-grounded), lifting Option B over Option A from 0.274 to 0.419 MRR, with
39 external validation on the ACI-Bench golden graph.

40 2 Related Work

41 **Clinical KG extraction.** Multi-agent and decomposed LLM pipelines extract entities and relations
42 from clinical text, typically separating an entity-extraction stage from a relation stage; the ACI-Bench
43 golden reference we validate against was built this way. We depart from this line by treating extraction
44 as a *draft* to be refined rather than a final artifact, and by learning the refiner self-supervised.

45 **Self-supervised and world models.** Joint-embedding predictive architectures (JEPA) and BYOL-
46 style latent prediction learn representations by predicting masked latents under an exponential-
47 moving-average (EMA) target with stop-gradient, avoiding contrastive collapse without negatives.
48 We adapt this objective to graphs as the pre-training stage of our refiner, yielding a single shared
49 encoder we treat as a world model over clinical state.

50 **Knowledge-graph completion.** Bilinear scoring functions such as DistMult provide an efficient
51 edge-recovery readout. We apply one over the *frozen* world-model encoder, trained with an InfoNCE
52 objective and type-matched negative sampling, so that recovery quality reflects the pre-trained
53 representation rather than a jointly-tuned decoder.

54 **Clinical language models.** Domain-pretrained bidirectional encoders (e.g. ModernBERT and its
55 clinical variant, Clinical-ModernBERT) provide note representations. Our contribution is not the note
56 encoder but *where* its output is injected into the graph.

57 3 Method: Clinical Graphs as Predictive World Models

58 We formulate clinical knowledge graph refinement as a world-modeling problem over structured
59 patient-state memory. The observation is a structured MIMIC-IV admission record together with its
60 source-faithful clinical narrative, the state is a typed clinical knowledge graph, and revision actions
61 correspond to retaining, reviewing, pruning, or proposing relations. The graph acts as an externalized
62 patient-state representation that organizes diagnoses, medications, procedures, microbiology, services,
63 and admission context into an auditable structure. Our system does not make autonomous clinical
64 decisions; it improves the structured representation used by downstream retrieval and decision-support
65 systems.

66 Figure 1 summarizes the pipeline. Panel A shows how structured MIMIC-IV records are converted
67 into patient-state graphs, combining deterministic table-derived edges with narrative-mediated in-
68 ferred clinical cross-links. Panel B shows Graph-JEPA training, where node features combine entity
69 text embeddings with localized note context, followed by masked patch pretraining and schema-aware
70 graph-revision fine-tuning. Panel C shows inference-time revision, where learned relation plausibility
71 and patch-level predictive consistency are combined before a schema guard assigns the final revision
72 action.

73 3.1 Consolidated clinical graph

74 Each admission becomes one *consolidated knowledge graph* fusing two evidence streams. A
75 *deterministic backbone* is read from the MIMIC-IV structured tables — patient → diagnoses,
76 medications, procedures, microbiology, and services (HAS_DIAGNOSIS, TAKES_MEDICATION,
77 UNDERWENT_PROCEDURE, MANAGED_BY_SERVICE, HAD_MICROBIOLOGY) — each at confidence 1.0.
78 Over this scaffold, four specialist agents propose the inferred relations the tables do not encode:
79 MANAGED_FOR (medication→diagnosis), CONFIRMS (procedure→diagnosis), COMPLICATED_BY
80 (diagnosis→diagnosis), and INDICATES (presenting symptom→diagnosis).

81 **Deterministic evidence scoring.** Every inferred edge receives a deterministic support vector
82 computed from model confidence, biomedical knowledge bases, ontology proximity, and note
83 provenance. These signals include RxNorm linkage, RxNav/MED-RT may-treat evidence, Hetionet

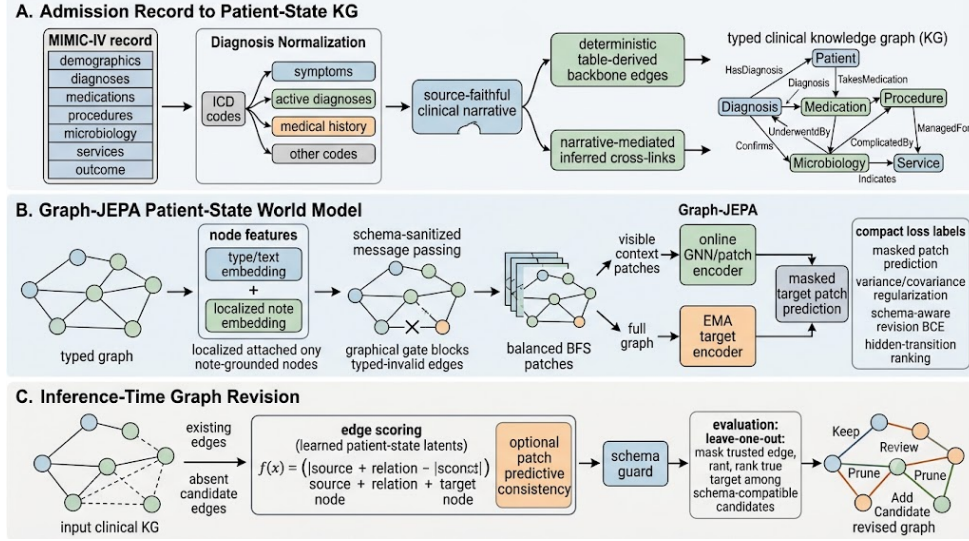


Figure 1: Overview of Clinical Graph-JEPA. (A) Staged extraction converts MIMIC-IV record into evidence-grounded patient-state graphs. (B) Graph-JEPA is pretrained with masked patch prediction and fine-tuned with schema-aware edge revision and ranking. (C) Inference combines edge plausibility, patch-level predictive consistency, and a schema guard to assign KEEP, REVIEW, PRUNE, or unverified-add actions.

84 treatment and disease-similarity relations, OMOP concept similarity and ancestor distance, and
85 note-grounding checks (`prov_in_note`, `prov_ratio`). The same evidence-scored graph is used for
86 both deployment options; the only difference between options is whether note features are provided
87 to the refiner.

88 3.2 Graph-JEPA world model

89 The refiner trains in two phases. *Phase 1* is self-supervised masked-latent prediction: a context
90 encoder embeds a masked graph and predicts the latents assigned by an EMA target encoder to the
91 hidden state. The encoder is a typed graph transformer over the consolidated graph. Node inputs
92 include a type embedding, a hashed-entity embedding, projected numeric features, and, only under
93 Option B, a localized note vector. *Phase 2* freezes the encoder and trains an edge-recovery readout
94 with an InfoNCE objective using $K=8$ type-matched negatives. This trains the world model to
95 recover graph-state relations from the surrounding patient-state memory rather than from labels or
96 clinical outcomes.

97 3.3 Option B: entity-grounded note injection

98 Option B augments the note-free graph state with note content when a discharge note is available.
99 The note is encoded once using Clinical-ModernBERT (Simonlee711/Clinical_ModernBERT;
100 768-dimensional mean-pooled embedding). We do *not* attach this vector to a global note node.
101 Instead, we inject it locally: the note vector is concatenated only to entities grounded by the note,
102 operationalized as nodes incident to a note-provenanced edge. Ungrounded entities receive a zero
103 note vector.

104 This entity-grounded placement is essential. A single admission-global note vector degrades recovery
105 because it is diffuse and largely redundant with the graph structure extracted from the note. Localizing
106 the same note representation onto the entities it discusses instead provides targeted context for relation
107 revision.

108 3.4 Deployment options A and B under partial notes

109 A deployable refiner must handle partial note availability: only 64.4% of admissions have a discharge
110 note. We therefore evaluate two options.

111 **Option A — note-free graph refiner.** Option A is the note-free Graph-JEPA refiner. It consumes
112 only the consolidated clinical graph: node type/text features, typed relations, schema information,
113 edge confidence, and deterministic edge-support metadata. It does not receive the discharge note or
114 any note embedding. During training, schema-valid trusted edges are treated as positive graph-state
115 transitions, while schema-valid corruptions, typed-invalid observed edges, reversed invalid triples,
116 and weak low-confidence edges provide negatives. A hidden-edge ranking objective further removes
117 trusted edges from message passing and trains the model to recover the true relation above hard type-
118 compatible alternatives. Because it requires no note at inference time, Option A applies uniformly to
119 all admissions.

120 **Option B — note-augmented refiner under partial notes.** Option B keeps the same graph,
121 schema, encoder, training losses, and evaluation protocol as Option A, but augments node
122 features when a note is available. The note is encoded once using Clinical-ModernBERT
123 (Simonlee711/Clinical_ModernBERT; 768-dimensional mean-pooled embedding) and injected
124 locally onto note-grounded entities. Nodes not grounded in the note receive a zero note vector. For
125 admissions without a note, Option B falls back to Option A, preserving full cohort coverage.

126 Thus, Option A is the fully deployable note-free refiner, while Option B asks whether localized note
127 context improves graph revision on the subset where notes exist. The two options differ only in the
128 note feature.

129 4 Results

130 4.1 Data and Cohort Construction

131 We use MIMIC-IV, a de-identified critical-care EHR. From the full set of 546,028 admissions
132 we identify 175,165 *eligible* patients — those with at least one admission meeting completeness
133 criteria (≥ 3 diagnoses, ≥ 3 medications, ≥ 1 service) — and draw a seed-fixed sample of 1,000
134 patients, retaining *all* of each patient’s admissions to preserve longitudinal structure. This yields
135 2,969 admissions (mean 3.0 per patient). Critically, a discharge note is present for only 1,912/2,969
136 (64.4%) of these admissions; 1,057 (35.6%) have none. At the patient level, 523 patients are fully
137 noted, 251 have no noted admission, and the remainder are partial. This partiality is the empirical
138 basis for the two-option design of Section 3.4.

139 4.2 Protocol

140 We evaluate by *leave-one-out* (LOO) edge recovery, the fair link-prediction protocol for a refiner: a
141 single edge is removed (with its inverse, to prevent leakage), the rest of the consolidated graph is kept
142 as context, and the model must recover the held-out edge by ranking the true target against same-type
143 candidates under a *filtered* protocol (other true tails of the query excluded). Ranking is deterministic
144 and reproducible. We report mean reciprocal rank (MRR) and Hits@ k overall and per relation, with
145 chance computed from candidate-set size. We additionally report a held-out batch-mask AUC and a
146 *non-obvious* AUC that excludes trivial patient-hub targets. The evaluation is explicitly of a *refiner*:
147 it improves an already-constructed draft and presupposes the consolidated graph as context, not a
148 cold-start predictor.

149 4.3 Internal results (MIMIC-IV held-out): Option A vs Option B

150 Table 1 is the central result. **Option A** (note-free) and **Option B** (note-augmented, on noted admis-
151 sions) are measured on identical data, split, seed, and *training recipe* (4,000 graphs; 3,200/400/400
152 split; $n=8,283$ LOO edges); they differ only in the note feature, so Δ is the pure effect of the note.

153 Option B improves *every* inferred relation, with the largest gains on INDICATES (the presenting-
154 symptom→diagnosis link the note narrates most directly, $0.373 \rightarrow 0.603$) and COMPLICATED_BY

Inferred relation	Option A (note-free)	Option B (note-augmented)	Δ
MANAGED_FOR	0.246	0.360	+0.114
INDICATES	0.373	0.603	+0.230
COMPLICATED_BY	0.273	0.471	+0.198
CONFIRMS	0.314	0.344	+0.030
Overall LOO MRR	0.274	0.419	+0.145

Table 1: Leave-one-out edge recovery (MRR), Option A vs Option B, on the four inferred relations. Identical recipe; only the note feature differs.

(0.273 $\rightarrow$ 0.471, $\approx 1.7\times$). The smallest gain is on CONFIRMS (+0.030), where Option A already recovers comparatively well (0.314). Crucially, Option A stays well above chance on every relation, which makes it a viable standalone configuration and the fallback inside Option B for note-less admissions.

Table 2 reports global metrics. Option B raises overall LOO Hits@1 (0.108 $\rightarrow$ 0.247) and Hits@10 (0.710 $\rightarrow$ 0.864) and, on held-out batch-mask scoring, the discriminating *non-obvious* AUC (0.712 $\rightarrow$ 0.800). The undifferentiated batch-mask AUC is saturated by trivial patient-hub edges and is not the meaningful measure; the non-obvious AUC and the LOO MRR are.

Metric	Option A	Option B
Batch-mask AUC	0.732	0.714
Non-obvious AUC	0.712	0.800
Batch-mask MRR	0.340	0.361
LOO MRR (overall)	0.274	0.419
LOO Hits@1	0.108	0.247
LOO Hits@10	0.710	0.864

Table 2: Global metrics. The note (Option B) helps on every refiner-relevant measure; the hub-saturated batch-mask AUC is not discriminating.

4.4 Ablation: note placement

Isolating the note feature on identical data confirms that *where* the note is placed is decisive (Table 3). A diffuse, admission-global note vector is *worse* than no note at all; localising the same vector onto note-grounded entities is what converts it into signal, and is the form used by Option B.

Configuration	Overall LOO MRR
Option A: note-free (no note)	0.274
Global NOTE node (diffuse note)	0.265
Option B: entity-grounded note	0.419

Table 3: Note-placement ablation. The same note vector helps only when localised onto the entities it grounds.

4.5 Where does the recovery come from? A context analysis

To attribute Option B’s recovery we isolate, per inferred relation, three context conditions: a *floor* (only the deterministic backbone present), a *cascade* (the backbone plus earlier inferred relations), and the full LOO *ceiling* (all other edges present). Table 4 shows that the deterministic backbone alone already supplies most of the recoverable context (e.g. INDICATES 0.467 at the floor vs. 0.603 at the ceiling), with the remaining headroom coming from the other inferred cross-links. This indicates the model exploits the free, certain structure first and the inferred relations second — consistent with the refiner framing.

Inferred relation	Floor (backbone)	Cascade	Ceiling (LOO)
MANAGED_FOR	0.291	0.291	0.360
CONFIRMS	0.282	0.289	0.344
COMPLICATED_BY	0.359	0.420	0.471
INDICATES	0.467	0.523	0.603

Table 4: Context analysis for Option B (MRR). The deterministic backbone (floor) provides most of the context; inferred cross-links add the remainder.

5 Discussion

Why entity-grounding works. The note and the graph carry overlapping information — the entities were extracted *from* the note — so a global note summary is largely redundant and adds variance. Placing the note on the specific entities it grounds turns it into a localised prior that distinguishes note-discussed entities from purely structural ones, which is exactly the distinction the inferred relations depend on. This is consistent with two further negative results we observed: feeding the per-edge evidence scores directly into the encoder also failed to help, and a separate global NOTE node hurt. Diffuse, graph-level features add little to a structure-first world model; localised, edge- or entity-specific signal is what helps.

Why two options. Reporting a single note-dependent model would overstate deployable accuracy, since a third of admissions have no note. Option A establishes the universally-available floor; Option B is the achievable ceiling where notes exist. The two-option framing makes the system honest about coverage while still capturing the note’s value.

6 Conclusion

Fawkes treats a noisy, LLM-drafted clinical KG as something to be *refined* rather than trusted. A Graph-JEPA world model, trained self-supervised over evidence-scored MIMIC-IV graphs, recovers the inferred clinical relations the structured tables omit. Because notes are only partially available, we report two deployment options: **Option A**, a note-free model covering every admission, and **Option B**, a note-augmented model used where a note exists with Option A as its fallback. Injecting note content locally, onto the entities it grounds, lifts Option B over Option A from 0.274 to 0.419 MRR under an identical recipe. Future work extends the longitudinal cohort, adds a graded edge-pruning policy, and replaces the mean-pooled note with token-level, entity-specific note attention.

References

- [1] A. Johnson et al. “MIMIC-IV, a freely accessible electronic health record dataset.” *Scientific Data*, 2023.
- [2] W. Yim et al. “ACI-Bench: a Novel Ambient Clinical Intelligence Dataset for Benchmarking Automatic Visit Note Generation.” *Scientific Data*, 2023.
- [3] M. Assran et al. “Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture (I-JEPA).” *CVPR*, 2023.
- [4] J.-B. Grill et al. “Bootstrap Your Own Latent (BYOL).” *NeurIPS*, 2020.
- [5] B. Yang et al. “Embedding Entities and Relations for Learning and Inference in Knowledge Bases (Dist-Mult).” *ICLR*, 2015.
- [6] A. van den Oord et al. “Representation Learning with Contrastive Predictive Coding (InfoNCE).” *arXiv:1807.03748*, 2018.
- [7] Y. Shi et al. “Masked Label Prediction: Unified Message Passing Model (TransformerConv).” *IJCAI*, 2021.
- [8] B. Warner et al. “Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder (ModernBERT).” *arXiv:2412.13663*, 2024.
- [9] D. Himmelstein et al. “Systematic integration of biomedical knowledge prioritizes drugs for repurposing (Hetionet).” *eLife*, 2017.

A Additional Method Details

A.1 MIMIC Graph Schema

For MIMIC-derived graphs, each admission graph is represented as $G_i = (V_i, E_i)$ with typed nodes and directed typed edges. The node type set is

$$\mathcal{T}_{\text{MIMIC}} = \{\text{PATIENT, DIAGNOSIS, MEDICATION, PROCEDURE, MICROBIOLOGY, SERVICE}\}. \quad (1)$$

When available, the schema can also include laboratory-test nodes, but in the MIMIC-IV extract used here laboratory values are not available and laboratory features are therefore not used as measured result nodes.

The relation set contains deterministic backbone relations and inferred clinical cross-links:

$$\mathcal{R}_{\text{MIMIC}} = \{\text{HAS_DIAGNOSIS, TAKES_MEDICATION, UNDERWENT_PROCEDURE, HAD_MICROBIOLOGY, MANAGED_BY_SERVICE, TARGETS_ORGANISM, MANAGED_FOR, CONFIRMS, COMPLICATED_BY, INDICATES}\}. \quad (2)$$

The deterministic backbone relations connect the patient node to observed entities in the structured record. Examples include

$$\text{PATIENT} \xrightarrow{\text{HAS_DIAGNOSIS}} \text{DIAGNOSIS},$$

$$\text{PATIENT} \xrightarrow{\text{TAKES_MEDICATION}} \text{MEDICATION},$$

and

$$\text{PATIENT} \xrightarrow{\text{UNDERWENT_PROCEDURE}} \text{PROCEDURE}.$$

Microbiology susceptibility data provide direct antibiotic–organism evidence, represented as

$$\text{MEDICATION} \xrightarrow{\text{TARGETS_ORGANISM}} \text{MICROBIOLOGY}.$$

The inferred cross-link schema includes clinically meaningful relations not stored directly as MIMIC relational columns:

$$\text{MEDICATION} \xrightarrow{\text{MANAGED_FOR}} \text{DIAGNOSIS},$$

$$\text{PROCEDURE} \xrightarrow{\text{CONFIRMS}} \text{DIAGNOSIS},$$

$$\text{DIAGNOSIS} \xrightarrow{\text{COMPLICATED_BY}} \text{DIAGNOSIS},$$

and

$$\text{DIAGNOSIS} \xrightarrow{\text{INDICATES}} \text{DIAGNOSIS}.$$

Presenting symptoms are represented as diagnosis-layer nodes with a presenting attribute, so symptom–diagnosis links use the same node type while preserving the symptom role in node metadata.

A.2 MIMIC Record Normalization

Each graph is built from a flattened MIMIC-IV admission record. The record contains patient and admission identifiers, demographics, admission and discharge metadata, diagnoses, medications, procedures, microbiology, services, and outcome fields. Diagnoses are partitioned into four clinically distinct buckets: presenting symptoms, active diagnoses, medical history, and other/external codes. This partition prevents historical or external-status codes from being treated as active admission diagnoses.

Medications retain dose, unit, route, and count metadata when available. Microbiology records retain specimen, organism, antibiotic, and susceptibility interpretation. Services and transfer-derived location information are retained as admission context. These fields provide the source of graph nodes and the metadata attached to nodes and edges.

244 A.3 Graph Construction

245 Given a normalized admission record r_i , graph construction first creates a deterministic node set

$$V_i = V_i^{\text{patient}} \cup V_i^{\text{dx}} \cup V_i^{\text{med}} \cup V_i^{\text{proc}} \cup V_i^{\text{micro}} \cup V_i^{\text{service}}. \quad (3)$$

246 The deterministic backbone edge set is

$$E_i^{\text{backbone}} = B(r_i), \quad (4)$$

247 where B adds direct table-derived relations such as diagnoses, medications, procedures, microbiology,
248 services, and susceptibility-supported antibiotic–organism links.

249 To infer clinical cross-links, the structured record is rendered into a source-faithful synthetic discharge-
250 style narrative $n_i = N(r_i)$. The narrative is constrained to mention only facts present in r_i . Relation-
251 specific extraction agents then operate over candidate pairs of existing graph nodes:

$$E_i^{\text{cross}} = A_{\text{managed}}(V_i, n_i) \cup A_{\text{confirms}}(V_i, n_i) \cup A_{\text{complicated}}(V_i, n_i) \cup A_{\text{indicates}}(V_i, n_i). \quad (5)$$

252 The final graph is

$$G_i = (V_i, E_i^{\text{backbone}} \cup E_i^{\text{cross}}). \quad (6)$$

253 The extraction agents are constrained to link only nodes already present in V_i . Candidate edges with
254 unmatched endpoints, invalid type signatures, self-loops, duplicate entries, or unsupported evidence
255 are removed during validation.

256 A.4 Provenance and Edge Support

257 Each edge stores its source node, target node, relation type, confidence, source field or evidence text,
258 and provenance metadata. Backbone edges are directly grounded in MIMIC tables and are assigned
259 full confidence. Inferred cross-links store cited evidence from the generated narrative and are further
260 annotated with deterministic support signals.

261 For medication–diagnosis relations, support includes biomedical semantic similarity and treatment
262 evidence from resources such as RxNorm/RxNav and Hetionet. Additional support features include
263 OMOP concept similarity or ontology proximity when available, as well as provenance checks
264 indicating whether the cited evidence appears in the narrative. These scores are retained as continuous
265 edge attributes rather than collapsed into hard-coded rules. This allows downstream models to learn
266 how to use support signals while preserving the original high-recall graph.

267 A.5 Graph Unit and Entity Identity

268 The training unit is an admission-level graph. Entities are grounded in the structured MIMIC record
269 before relation extraction, and inferred edges are restricted to those existing entities. Therefore, the
270 current MIMIC pipeline does not require corpus-level embedding-based entity resolution to construct
271 the training graphs. Node normalization is performed within the admission record, and node metadata
272 retains the original MIMIC-derived names, codes, and source fields needed for provenance.

273 A.6 Node and Note Features

274 Each node is represented by a base embedding of its type and canonical text:

$$b_v = f_\phi(\tau(v), \text{text}(v)). \quad (7)$$

275 When an admission-level note embedding n_i is available, it is localized to nodes grounded in the note.
276 Let $a_v \in \{0, 1\}$ indicate whether node v is grounded. The final node feature is

$$x_v = [b_v, a_v n_i]. \quad (8)$$

277 If no note embedding is available, the note component is zero.

278 By default, a node is considered grounded when it participates in an edge whose provenance check
279 indicates support in the note. We also support grounding by surface-name match to the note, or
280 assigning the note embedding to all non-patient nodes. This keeps note context localized to clinically
281 grounded entities rather than broadcasting it uniformly to the whole graph.

282 A.7 GNN Encoder, Graph Patching, and Target Encoding

283 The Graph-JEPA encoder uses typed message passing over the schema-sanitized message graph.
 284 Relation labels are embedded and used as edge attributes in the GNN. The final node latent is

$$z_v = W_o h_v^{(L)}. \quad (9)$$

285 The graph is partitioned into K local patches using balanced multi-source BFS. Patch representations
 286 are mean-pooled node latents:

$$q_j = \frac{1}{|P_j|} \sum_{v \in P_j} z_v. \quad (10)$$

287 Patch positional features include relative patch size, patch degree, and random-walk return features
 288 on the coarsened patch graph.

289 For each graph, context patches C and target patches T are sampled. The online encoder receives
 290 visible context patches and learned mask content for hidden patches. The EMA target encoder
 291 receives the full graph and is updated by

$$\bar{\theta} \leftarrow m\bar{\theta} + (1 - m)\theta. \quad (11)$$

292 For each $j \in T$, the predictor maps the context patch state and positional features to $\hat{q}_j$.

293 A.8 Revision Supervision and Loss

294 Let $\mathcal{D}^+$ be trusted observed edges that satisfy the typed schema:

$$\mathcal{D}^+ = \{(u, r, v) \in E : (\tau(u), r, \tau(v)) \in \mathcal{S} \text{ and } e \text{ is trusted}\}. \quad (12)$$

295 When confidence supervision is enabled, high-confidence generated edges are trusted positives,
 296 low-confidence generated edges are weak negatives, and intermediate-confidence edges are ignored.
 297 Without this supervision, all schema-valid observed edges are treated as trusted positives.

298 Negative examples combine schema-valid corruptions from the same graph, observed schema-invalid
 299 edges, reversed triples whose direction violates the schema, and weak generated negatives. The
 300 revision loss is relation-balanced binary cross-entropy:

$$\mathcal{L}_{\text{rev}} = \frac{1}{|\mathcal{R}_{\text{obs}}|} \sum_{r \in \mathcal{R}_{\text{obs}}} \frac{1}{|\mathcal{D}_r|} \sum_{(u, r, v, y) \in \mathcal{D}_r} \text{BCE}(s_\theta(u, r, v), y). \quad (13)$$

301 Weak generated negatives may receive a lower weight than schema negatives.

302 A.9 Hidden-Edge Candidate Ranking

303 For candidate-ranking fine-tuning, a subset of trusted schema-valid edges is hidden from message
 304 passing. For each hidden edge $e_i = (u_i, r_i, v_i)$, hard distractors are sampled from the same graph by
 305 replacing the target with another schema-valid target, replacing the source with another schema-valid
 306 source, or replacing the relation with another schema-valid relation for the same endpoints. Let
 307 $\mathcal{C}_i = \{e_i\} \cup \mathcal{N}_i$ be the true edge and its distractors. The ranking loss is

$$\mathcal{L}_{\text{rank}} = -\frac{1}{|H|} \sum_{i \in H} \log \frac{\exp(s_\theta(e_i)/T)}{\sum_{e \in \mathcal{C}_i} \exp(s_\theta(e)/T)}. \quad (14)$$

308 A.10 Inference-Time Scoring

309 For an edge $e = (u, r, v)$, the edge head gives $p_\theta(u, r, v)$. The structural score masks the target patch
 310 $j(v)$ and computes normalized prediction energy,

$$\epsilon_{j(v)} = \frac{\|g_\theta([c_{j(v)}, p_{j(v)}]) - \bar{q}_{j(v)}\|_2}{\sqrt{d}}, \quad (15)$$

311 then converts it to

$$s_{\text{struct}}(e) = \exp(-\epsilon_{j(v)}/T). \quad (16)$$

312 The revision score $J(e)$ combines edge plausibility and structural consistency. Typed-invalid edges
313 are capped at a fixed invalid-edge score and flagged as inconsistent.

314 For leave-one-out recovery, each trusted edge is removed from the message graph, the source and
315 relation are held fixed, and the true target is ranked against schema-compatible candidates. Metrics
316 are computed with MRR and Hits@ k .

317 **A.11 Candidate Edge Generation**

318 Candidate generation enumerates absent schema-valid triples

$$\mathcal{A}(G) = \{(u, r, v) : (u, r, v) \notin E, (\tau(u), r, \tau(v)) \in \mathcal{S}\}. \quad (17)$$

319 Candidates can be filtered by encounter context, negation status, disabled relations, and relation-
320 specific thresholds. High-scoring candidates are added only as unverified suggestions without
321 evidence. Thus candidate generation is an edge-recovery proposal mechanism, not a claim of clinical
322 truth.